
Human Detection in Drone Imagery for Wilderness SAR: A Benchmark on the WiSARD Dataset

Prachee Nanda

SYDE 522

University of Waterloo

Student ID: 20939815

p4nanda@uwaterloo.ca

Lavanya Yadav

SYDE 522

University of Waterloo

Student ID: 20939815

lyadav@uwaterloo.ca

Abstract

Wilderness search and rescue operations are hindered by the manual analysis of vast amounts of drone imagery, delaying critical rescue efforts and increasing the risk for missing individuals. Thus, in this paper, we propose a lightweight human detection approach to automate drone footage inspection. We use the WiSARD dataset, which contains labeled visual and thermal images collected from UAV flights in various conditions. We benchmarked five rigorous classifiers on both forms of images to recognize humans: Support Vector Machine (SVM), Random Forest (RF), Convolutional Neural Network (CNN), Multi layer Perceptron (MLP) based on four feature extraction techniques: Histogram of Oriented Gradients (HOG), Canny Edge Detection, and an approach that uses thermal images as binary masks to filter and enhance visual images. We further fine-tuned the hyper parameters of the best-performing classification algorithms, with the best performing model achieving 93% accuracy.

1 Introduction

Wilderness Search and Rescue (WiSAR) refers to the process of locating and providing aid to people who are lost in remote outdoor environments. These operations often take place in areas with challenging terrain, with limited access to resources and harsh environmental conditions. Traditional search and rescue methods rely on human effort, which can make the process slow, expensive, and risky. Due to these difficulties, there is a growing need for new technologies that can make these missions more effective.

Currently, the use of unmanned aerial vehicles (UAVs) equipped are increasingly being used to improve SAR operations by providing real-time aerial imagery and thermal data. However, analyzing this vast amounts of drone imagery remains a major bottleneck, as identifying subtle visual clues indicating human presence is still largely performed manually. To address this challenge, this work contributes to SAR operations by exploring computer vision methods specifically tailored for wilderness environments.

2 Dataset

<https://sites.google.com/uw.edu/wisard/?pli=1>

2.1 Dataset Overview

The *WiSARD dataset* consists of visual and thermal images collected via drone flights over wilderness environments across Washington, USA. Developed by the Autonomous Flight Systems Lab at the

University of Washington’s Aeronautics and Astronautics Department, this dataset specifically targets human detection in dense, rugged wilderness environments — an area underserved by existing datasets.

Unlike large-scale object detection datasets such as VisDrone, which primarily focus on urban and rural settings, the WiSARD dataset is tailored for SAR scenarios in complex wilderness terrains. Furthermore, compared to existing SAR datasets like HERIDAL that offers only visual images and limited in scale, or UMA-SAR that lacks annotated ground truth data, the WiSARD dataset includes both visual and thermal images with precise bounding box annotations for human presence. The dataset includes images captured under diverse conditions — such as varying terrains, lighting, and weather, which helps improve model generalization and robustness by exposing models to real-world variability and enabling them to better handle unseen scenarios in WiSAR applications.

The WiSARD dataset consists of three main subsets of data: visual (with 26,862 labeled visual images), thermal (with 29,989 labeled Long Wave Infrared (thermal) images) and multi-modal (with 15,453 temporally synchronized visual-thermal image pairs) set of images. Due to the images being very large and computational constraints, for the purposes of this paper, we work with a smaller set (about 600 images) of synchronized and annotated visual-thermal image pairs provided (from the dataset source).

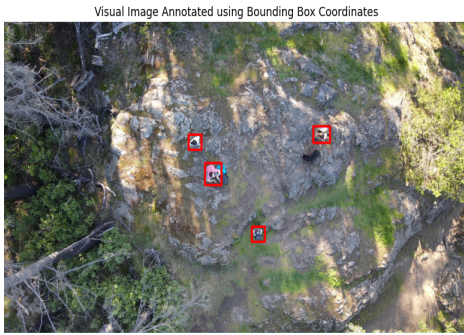


Figure 1: Visual Image - Annotated with Bounding Box Coordinates from the Dataset

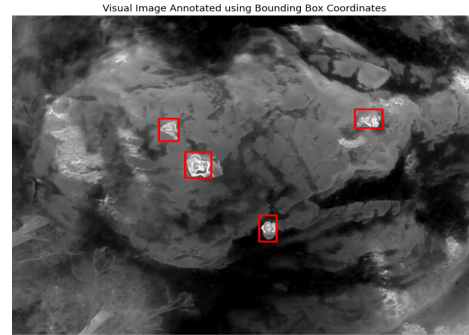


Figure 2: Thermal Image - Annotated with Bounding Box Coordinates from the Dataset

3 Data Pre-Processing

The data underwent a multi-step preprocessing pipeline to ensure sufficient model training.

3.1 Image Segmentation and Pre-processing

The raw images in the WiSARD dataset are too large for efficient processing with visual images sized at 3840x2160 and thermal images sized at 640x512. To address this, our first step involved breaking down the large images into smaller tile-sized images, which increased the number of images being used for model training and validation to about 10560 visual and 1584 thermal images. Through experimentation, we found that too small of a tile size leads to a loss of critical visual information, while too large of a tile size leads increased processing time. Upon evaluating the images from the dataset, we discovered that the size of our objects of interest (humans) is roughly 128x128 pixels in visual images and 64x64 pixels in thermal images. Based on this information, we chose a 256x256 and 512x512 tile size respectively to ensure sufficient context while maintaining computational efficiency on both subsets.

3.2 Semantic Segmentation

The bounding box annotations provided were used to localize and label regions of interest (ROI) within the images. Given the task of detecting human presence, the image tiles were adjusted to focus on extracting regions containing humans, while the remaining areas were designated as background. Binary labels were assigned to the images: Class 0 for "No Human Present" and Class 1 for "Human

Present." This semantic labeling enables supervised learning algorithms to effectively differentiate between positive (human) and negative (background) samples.

3.3 Feature Extraction

Feature extraction is a critical step in the detection pipeline, where meaningful representations are derived from raw image data to emphasize essential patterns and structures. In this work, we employ a range of feature extraction techniques, including *Histogram of Oriented Gradients (HOG)*, *Canny Edge Detection*, and an approach that uses thermal images as binary masks for the visual images and then *HOG* is used to extract features from the masked image to filter and enhance visual images.

3.3.1 Histogram of Oriented Gradients (HOG)

HOG is a widely used feature descriptor for human body detection, first introduced by Dalal and Triggs. It divides the image into small regions, computes the gradient directions and magnitudes for each pixel, and groups them into orientation bins to capture edge patterns. In this implementation, to reduce the impact of lighting variations, the image is first converted to grayscale mode, normalized to the range $[0, 1]$, and resized to a fixed dimension of 128×128 pixels. We further divided the grayscale image into cells of 8×8 pixels and used 9 orientation bins to capture the gradient information within each cell. The resulting local histograms were then flattened into a single feature vector, effectively encoding edge and gradient information for downstream human detection tasks.



Figure 3: Original Visual Image (Grayscale)

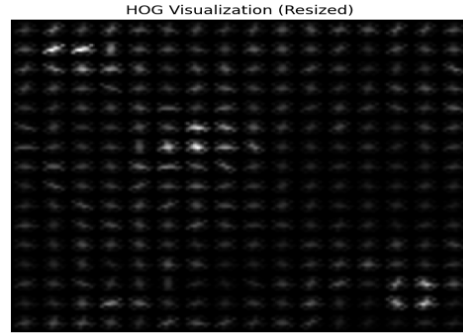


Figure 4: HOG visualization

3.3.2 Canny Edge Detection

Canny Edge Detection identifies boundaries in an image by detecting areas with rapid intensity changes. It is known for its precision and effectiveness in identifying edges while minimizing noise. The gradient magnitude and orientation are calculated for each pixel to identify areas of rapid intensity change. Edge pixels are thinned to single pixel width by suppressing pixels that are not local maxima along the gradient direction. It applies two thresholds against gradient magnitudes of pixels to classify edges as strong or weak. By connecting these edges together, we extract the boundary of figures in the object. In this paper, thresholds of 100 and 200 were found to perform well on images in the WiSARD dataset. To reduce the impact of lighting variations and noise, the image was first converted to grayscale, normalized to the range $[0, 1]$, and resized to a consistent dimension of 128×128 pixels. A Gaussian blur is then applied to smooth the image, reducing noise that can cause false edge detection.

3.3.3 Utilizing thermal images as binary masks to filter and enhance visual images

This method involves using the thermal modality as a mask for the corresponding visual images. We attempt to explore methods for extracting more informative features by combining complementary information from both modalities. The visual images are 3-channel (RGB - Red, Green, Blue) images with pixel intensities in the range $[0, 255]$ for each of the three channels. The thermal images, on the other hand, are single-channel grayscale images with pixel intensity in the range $[0, 255]$ (with 0 being black and 255 being white), where higher values correspond to stronger heat signatures, indicating potential human presence. To integrate the thermal images as a mask, a bitwise AND

operation is applied between the visual image and the thermal image at each pixel coordinate (x, y) expressed as:

$$\text{Result}(x, y) = \text{Visual}(x, y) \wedge \text{Mask}(x, y)$$

where the logical AND operation is applied between the visual image’s pixel values and the thermal image’s pixel values. If the thermal mask’s pixel value is 0, the corresponding visual pixel is zeroed out; if it is 255, the visual pixel remains unchanged. This enhances regions with strong thermal signatures (areas of interest) while filtering out irrelevant regions. The features from the masked image are then extracted using Canny Edge Detection since it can highlight object boundaries efficiently without adding extra abstraction.

4 Analyzing the Extracted Features

Understanding the structure of the dataset is critical for selecting appropriate machine learning techniques for the classification task at hand. To gain insights into the distribution of the extracted features, we applied Principal Component Analysis (PCA) and t-Distributed Stochastic Neighbor Embedding (t-SNE) to reduce the dimensionality of the feature space to two dimensions for visualization purposes.

The resulting plots provide a visualization of the extracted features using PCA and t-SNE. The PCA plot shows that the two classes (represented by yellow and purple points) exhibit significant overlap in the projected two-dimensional space, indicating that the data is not linearly separable. Similarly, the t-SNE visualization highlights the complex structure of the feature relationships, revealing clusters with considerable intermingling of the two classes. These observations emphasize the need for more sophisticated, non-linear classifiers capable of capturing the intricate patterns within the data.

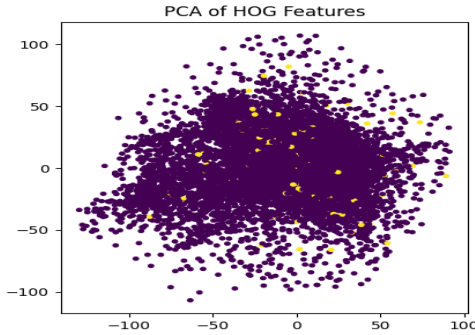


Figure 5: PCA plot of extracted features projected into two dimensions.

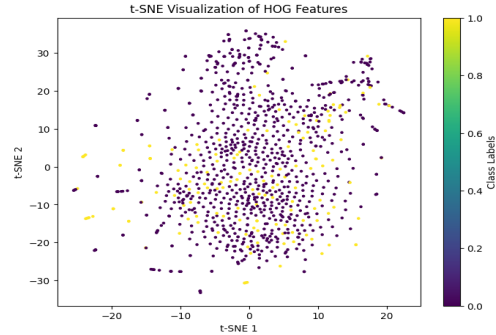


Figure 6: t-SNE visualization of the extracted features.

5 Model Selection

In this paper, we utilize *four robust classification algorithms namely SVM, RF, CNN and MLP* for classifying image segments into positive (human) and negative (background) classes. We have tested the models on features obtained from the thermal and visual datasets using all aforementioned feature extraction techniques.

5.1 Support Vector Machine (SVM)

SVM is a supervised learning algorithm used in machine learning to solve classification and regression tasks, especially for binary classification problems. It classifies data by finding an optimal line/hyperplane that maximizes the distance between each class. SVM with non-linear kernels (e.g. RBF, polynomial etc.) is able to map non-linear data into a higher-dimensional space where it becomes linearly separable, which makes it suitable for the purpose of this paper. For regularization of the model, we use class weight balancing to prevent the model from being biased towards the majority class, given the large number of Class 0 images compared to Class 1 images. By setting `class_weight='balanced'`, the algorithm adjusts the penalty for misclassifications based on the

Table 1: Model Performance for Visual Images using HOG and Canny Edge for Feature Detection

Model	Feature	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
SVM	HOG	96.00	96.00	96.00	96.00
	Canny Edge	95.00	95.00	95.00	94.00
MLP	HOG	95.00	95.00	95.00	95.00
	Canny Edge	94.89	95.00	95.00	94.00
CNN	HOG	96.00	96.00	96.00	95.00
	Canny Edge	95.74	95.00	96.00	95.00
RF	HOG	96.00	96.00	96.00	95.00
	Canny Edge	91.51	92.00	92.00	91.00

Table 2: Model Performance for Thermal Images using HOG and Canny Edge for Feature Detection

Model	Feature	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
SVM	HOG	88.64	89.00	89.00	88.00
	Canny Edge	85.17	85.00	85.00	84.00
MLP	HOG	88.64	88.00	89.00	88.00
	Canny Edge	83.60	85.00	85.00	84.00
CNN	HOG	91.17	91.00	91.00	91.00
	Canny Edge	86.00	85.00	86.00	86.00
RF	HOG	86.12	88.00	89.00	88.00
	Canny Edge	81.39	84.00	81.00	78.00

Table 3: Model Performance using Thermal Images as a Binary Mask and Extracting Features from the Masked Image using Canny Edge Detection

Model	Feature	Accuracy (%)	Precision (%)	Recall (%)	F1-Score (%)
SVM	Thermal images as binary masks	81.00	81.00	81.00	81.00
MLP	Thermal images as binary masks	94.37	100.00	100.00	100.00
CNN	Thermal images as binary masks	92.68	100.00	99.29	99.77
RF	Thermal images as binary masks	78.00	95.45	91.30	93.33

class frequencies. This ensures that the minority class (Class 1 - human) is not overshadowed by the majority class during training.

The results from the model training and testing are tabulated in Table 1 and 2.

5.2 Random Forest (RF)

Random Forest is an ensemble learning method that is primarily used for regression and classification tasks. It effectively handles non-linear data by combining multiple decision trees trained on bootstrapped subsets of the data (known as the bagging method) along with feature sampling which reduces variance. The choice of this model is motivated by its ability to model complex, non-linear relationships to high-dimensional data. Similar to the SVM model, class weights help with regularization in this model as well by addressing class imbalance which ensures the model generalizes better across all classes. This prevents the model from over-fitting to the majority class

and promotes fairer decision splits across all trees.
The results from the model training and testing are tabulated in Table 1 and 2.

5.3 Multi-Layer Perceptron (MLP)

A Multi-Layer Perceptron is a feedforward neural network that consists of an input layer, one or more hidden layers with non-linear activation functions (e.g. ReLU, Sigmoid etc.), and an output layer. This model is well-suited for non-linear data because the hidden layers are able to model non-linear relationships by combining features in hierarchical ways, allowing it to learn intricate patterns. To ensure generalization, early stopping monitors the performance of the model on the validation set and halts training when the validation performance ceases to improve for a specified number of epochs. This prevents the model from overfitting to the training data.

The results from the model training and testing are tabulated in Table 1 and 2.

5.4 Convolutional Neural Network (CNN)

A CNN is a deep learning model designed to process spatially structured data using convolutional layers to automatically extract hierarchical features. CNNs are appropriate for non-linear data with high-dimensional features because of their ability to capture patterns like edges, textures and complex shapes while maintaining spatial relationships within the input. For the baseline model used here, the architecture consists of two convolutional layers, followed by max-pooling operations, a fully connected dense layer, and a dropout layer to prevent overfitting. To address the challenge of class imbalance present in the dataset, we use class weights during model training, which assigns higher importance to underrepresented classes. Additionally, resampling methods like oversampling the minority class was applied to the training dataset before feeding it into the CNN.

The results from the model training and testing are tabulated in Table 1 and 2.

5.5 Multi-Modal Early Fusion - Combining RGB and Thermal Modalities

While feature extraction methods seem to work quite well, we suspected that there is still a slight possibility for them to fail in capturing cross-modal relationships between RGB and thermal inputs. So, in this approach, we employed early fusion to directly combine RGB and thermal images as input to a CNN, bypassing the traditional feature extraction methods described earlier in the paper, to test how well it would perform. The key advantage of early fusion lies in its ability to retain the full pixel-level information from both modalities, enabling the model to identify subtle patterns and relationships across RGB and thermal inputs. RGB images provide rich visual details but may underperform in low-light or obscured conditions, while thermal images capture heat signatures robust to lighting variations but lack textural information. By concatenating RGB (3-channel) and thermal (1-channel) images into a 4-channel input tensor that is then passed as input to the CNN, the model leverages both modalities to learn correlated features. To prevent the model from overfitting, we used regularization techniques such as dropout and batch normalization. A dropout rate of 0.5 was applied after the fully connected layer to encourage generalization by randomly deactivating neurons during training. Additionally, pixel-wise scaling (dividing inputs by 255) normalized the image data to a standard range [0, 1], stabilizing the training process, improving gradient consistency, and enabling faster convergence.

5.6 Extra: Using Thermal Images as a Binary Mask and Using the Masked Image Directly as an Input to a CNN

As described earlier in the paper, one of the methods we employed for feature extraction included using the thermal images as a binary mask for the visual images and generating a masked image. The features for this masked image were then extracted using Canny Edge Detection and passed as inputs to the different models mentioned above (results in Table 3). However, we were curious to see how a CNN would perform if we passed in the masked image directly as the input, in an attempt allow the model to learn spatial and contextual features automatically, rather than relying on prior feature extraction. The baseline CNN consists of three convolutional layers with 32, 64, and 128 filters of size 3×3 , each followed by 2×2 max-pooling. A dense layer with 128 neurons and a dropout rate of 0.5 is added before the final sigmoid layer for binary classification. The model is trained using the

Adam optimizer, binary cross-entropy loss, a batch size of 128, and 20 epochs. It performed decently, with a validation accuracy of 90.09%.

6 Hyperparameter Tuning

All of the models described above have been tested for one set of hyperparameters, with a goal of narrowing down what models give the best performance for the different types of data. Using the results in Table 1, Table 2 and Table 3, we now perform hyperparameter tuning for each of the best-performing models for each type of image and each method of feature extraction.

6.1 CNN Hyperparameter Tuning for HOG-Extracted Features for Thermal-Only Data (Image 1, Figure 7)

Hyperparameter tuning was performed using a manual grid search approach to identify the optimal combination of model parameters. The grid included variations in the number of filters for the first and second convolutional layers ($\text{filters_1} = [16, 32]$, $\text{filters_2} = [32, 64]$), dropout rates ($\text{dropout_rate} = [0.3, 0.5]$), batch sizes ($\text{batch_size} = [16, 32]$), and the number of training epochs ($\text{epochs} = [10]$). An early stopping mechanism was used to avoid overfitting by halting training after three consecutive epochs without improvement in validation loss. The best-performing model utilized the following hyperparameters: $\text{filters_1} = 32$, $\text{filters_2} = 64$, $\text{dropout_rate} = 0.3$, $\text{batch_size} = 32$, and $\text{epochs} = 10$, achieving a test set accuracy of 90.22%.

6.2 CNN Hyperparameter Tuning for HOG-Extracted Features for Visual-Only Data (Image 2, Figure 7)

Hyperparameter tuning was conducted using the same manual grid search approach and parameters as in the thermal-only data case. The results of the grid search identified the following best-performing model configuration: ($\text{filters_1} = 32$, $\text{filters_2} = 64$, $\text{dropout_rate} = 0.3$, $\text{batch_size} = 16$, and $\text{epochs} = 10$). With these optimal hyperparameters, the model achieved a test set accuracy of 96.13%, which is a slight improvement from the baseline model's accuracy.

6.3 CNN Hyperparameter Tuning for Canny Edge Detection for Thermal-Only Data (Image 3, Figure 7)

Hyperparameter tuning was conducted using a manual grid search approach to determine the optimal configuration of model parameters. The grid included variations in the number of filters for the first and second convolutional layers ($\text{filters_1} = [16, 32]$, $\text{filters_2} = [32, 64]$), the dropout rate ($\text{dropout_rate} = [0.3, 0.5]$), the batch size ($\text{batch_size} = [16, 32]$), and the number of training epochs ($\text{epochs} = [10, 20]$). The best-performing model utilized the following hyperparameters: $\text{filters_1} = 16$, $\text{filters_2} = 32$, $\text{dropout_rate} = 0.3$, $\text{batch_size} = 16$, and $\text{epochs} = 20$, achieving a test set accuracy of 86.44%.

6.4 CNN Hyperparameter Tuning for Canny Edge Detection for Visual-Only Data (Image 4, Figure 7)

Hyperparameter tuning was conducted using a manual grid search approach to determine the optimal configuration of model parameters. The grid included variations in the number of filters for the first and second convolutional layers ($\text{filters_1} = [16, 32]$, $\text{filters_2} = [32, 64]$), the dropout rate ($\text{dropout_rate} = [0.3, 0.5]$), the batch size ($\text{batch_size} = [32, 64]$), and the number of training epochs ($\text{epochs} = [10]$). The best-performing model utilized the following hyperparameters: $\text{filters_1} = 16$, $\text{filters_2} = 32$, $\text{dropout_rate} = 0.3$, $\text{batch_size} = 64$, and $\text{epochs} = 10$, achieving a test set accuracy of 96.31%.

6.5 CNN Hyperparameter Tuning for Using Thermal Images as a Binary Mask and Using the Masked Images as Direct Input to a CNN (Image 5, Figure 7)

For the CNN model that directly uses the masked images as input, hyperparameter tuning was performed using a manual grid search to identify the optimal configuration. The grid included

variations in the number of filters for the first, second, and third convolutional layers (filters_1 = [16, 32], filters_2 = [32, 64], filters_3 = [64, 128]), the batch size (batch_size = [32, 64]), and the number of training epochs (epochs = [10]). A dropout rate of 0.3 was applied after the fully connected layer to prevent overfitting, and early stopping with a patience of 3 epochs was used to stabilize training. The best-performing model had the following hyperparameters: filters_1 = 16, filters_2 = 32, filters_3 = 128, dropout_rate = 0.3, batch_size = 32, and epochs = 10 and achieved a validation accuracy of 88.68%

6.6 CNN Hyperparameter Tuning for Early Fusion of RGB and Thermal Images (Image 6, Figure 7)

For the CNN model utilizing early fusion of RGB and thermal images, hyperparameter tuning was conducted using a manual grid search to determine the optimal configuration of model parameters. The grid explored variations in the number of filters for the three convolutional layers (filters_1 = [16, 32], filters_2 = [32, 64], filters_3 = [64, 128]), dropout rate (dropout_rate = [0.3]), batch size (batch_size = [32, 64]), and the number of epochs (epochs = [10]). To ensure valid configurations, only filter combinations satisfying filters_1 < filters_2 < filters_3 were evaluated. The best-performing model had the following hyperparameters: filters_1 = 16, filters_2 = 32, filters_3 = 128, dropout_rate = 0.3, batch_size = 64, and epochs = 10 and achieved a validation accuracy of 91.98%.

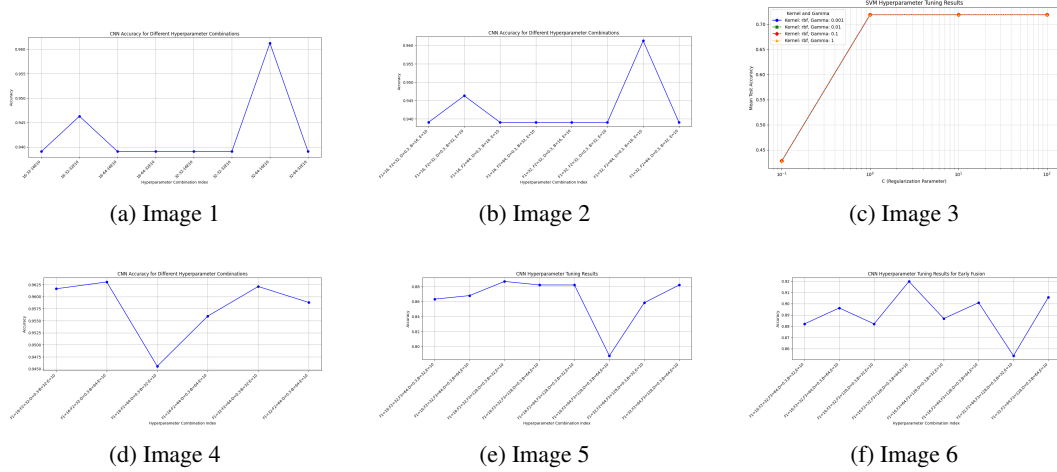


Figure 7: Graphs from Hyperparameter Tuning (x-axis = parameters, y-axis = accuracy)

7 Final Models and Conclusion

The final models and their optimal hyperparameters identified through manual grid search demonstrate the effectiveness of each model for the task at hand. Below are the final models for each data modality and feature extraction method:

- **CNN for HOG-extracted features (thermal-only data)**
Hyperparameters: filters_1 = 32, filters_2 = 64, dropout_rate = 0.3, batch_size = 32, epochs = 10
Accuracy: 90.22%
- **CNN for HOG-extracted features (visual-only data)**
Hyperparameters: filters_1 = 32, filters_2 = 64, dropout_rate = 0.3, batch_size = 16, epochs = 10
Accuracy: 96.13%
- **CNN for Canny edge-extracted features (thermal-only data)**
Hyperparameters: filters_1 = 16, filters_2 = 32, dropout_rate = 0.3, batch_size = 16, epochs = 20
Accuracy: 86.44%

- **CNN for Canny edge-extracted features (visual-only data)**
Hyperparameters: filters_1 = 16, filters_2 = 32, dropout_rate = 0.3, batch_size = 64, epochs = 10
Accuracy: 96.31%
- **CNN for using thermal images as a binary mask and using the masked images as direct input to a CNN**
Hyperparameters: filters_1 = 16, filters_2 = 32, filters_3 = 128, dropout_rate = 0.3, batch_size = 32, epochs = 10
Accuracy: 88.68%
- **CNN for Early Fusion of RGB and thermal images**
Hyperparameters: filters_1 = 16, filters_2 = 32, filters_3 = 128, dropout_rate = 0.3, batch_size = 64, epochs = 10
Accuracy: 91.98%

The results highlight **CNN models** as the most effective for human detection in both visual and thermal data. For visual-only data, the **CNN with HOG-extracted features** and the **CNN with Canny edge-extracted features** seem to achieve similar accuracies of approximately (**96%**). For thermal-only data, the **CNN with HOG-extracted features** achieved an accuracy of (**90.22%**) while the **CNN with Canny edge-extracted features** only achieved an accuracy of (**86.44%**). The **early fusion CNN** combining RGB and thermal data achieved an accuracy of **91.98%**, showcasing the benefits of multi-modal integration. These findings suggest that CNN models, particularly with HOG features or fused multi-modal inputs, are ideal for deployment in camera-equipped UAVs to enhance SAR operations by ensuring more efficient human detection in wilderness environments.

8 Future Improvements

8.1 Larger Dataset

Due to computational limitations, a smaller subset of the original dataset was used in this study. However, initial trials indicated that increasing the dataset size, while time-consuming, led to improved model accuracy. This improvement was largely attributed to the inclusion of more Class 1 (human) images, which helped address the class imbalance and allowed the models to better learn the distinguishing features of human presence in the data. Future work will focus on leveraging the full dataset to further enhance the model's performance and generalizability.

8.2 Transfer Learning

In future work, transfer learning can be explored to improve feature extraction by leveraging pre-trained Convolutional Neural Networks (CNNs) such as ImageNet models (e.g., VGG16, ResNet, or Inception). These models are trained on large-scale datasets and have already learned rich and hierarchical feature representations. By using these pre-trained networks as feature extractors, the lower layers can effectively capture edges, textures, and shapes, while higher layers can extract more complex and abstract features. Fine-tuning these models on the dataset can further adapt them to detect humans in the wilderness environment for better performance.

9 References

- [1] Broyles, D., Hayner, C., & Leung, K. (2022). WiSARD: A Labeled Visual and Thermal Image Dataset for Wilderness Search and Rescue. In IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS).
- [2] Martinez-Esteso, J. P., Castellanos, F. J., Calvo-Zaragoza, J., & Gallego, A. J. (2024, November 12). Maritime search and rescue missions with Aerial Images: A Survey. arXiv.org. <https://arxiv.org/abs/2411.07649>
- [3] HERIDAL Database. Ipsar. (n.d.). <http://ipsar.fesb.unist.hr/HERIDAL>